Il pulsante: digitalRead e if su Arduino

Lezione 11 — 1MAK, SST-INF-01c — Ripasso dell'if, leggere un pulsante, il circuito in Tinkercad, errori frequenti, esercizio con quattro LED

13/11/2026

1 Da dove ripartiamo: l'if

Nella lezione precedente (A10) abbiamo dato ad Arduino la possibilità di **prendere decisioni**. Lo strumento è l'istruzione `if`, che in inglese vuol dire "**se**": *se* succede qualcosa, *allora* esegui qualcos'altro.

Definizione

Struttura dell'if. Si scrive `if`, poi tra **parentesi tonde** la **condizione**; sotto la "i" si apre la **graffa**, e più in giù la graffa si chiude. Graffa aperta e graffa chiusa **delimitano** una porzione di codice, cioè un gruppo di **istruzioni** (comandi) che diamo alla macchina.
La **condizione** è un'espressione che può dare solo due risultati: **vero** o **falso**.

- Se è **vera**, Arduino **esegue** il codice tra le graffe (per esempio un `digitalWrite`, o tutto quello che impareremo nelle prossime settimane).
- Se è **falsa**, lo **salta a piè pari**: salta tutto quel pezzo di codice e va avanti con quello che c'è scritto dopo.

```
if (condizione)
{
  // qui il codice da eseguire se la condizione e' vera
}
```

1.1 Un esempio con due variabili

Una condizione può essere `a > b` ("**a maggiore di b**"). Abbiamo due variabili: `a` vale **5** e `b` vale **3**.

- 5 è maggiore di 3? **Sì**: la condizione è vera e Arduino **esegue** il codice tra le graffe.
- Se per qualche motivo la variabile cambia e `a`, invece di 5, vale **1**: 1 non è maggiore di 3, la condizione diventa **falsa** e Arduino **smette** di eseguire quel codice.

Esempio

L'ombrello. È come dire: *se piove, apro l'ombrello*. Nel momento in cui smette di piovere, posso chiuderlo. L'azione di aprire l'ombrello la decido proprio con un `if`: "se piove", e tra le graffe il pezzo di codice che apre l'ombrello. Da un'altra parte posso scrivere "se **non** piove" (la condizione **contraria**), e tra le graffe il codice che chiude l'ombrello. Allo stesso modo: *se ho fame, mangio*. Tutte le decisioni della macchina si costruiscono così.

1.2 Variabili che dipendono dall'esterno

Finora nei nostri programmi le variabili `a` e `b` le abbiamo **decise noi** all'inizio, e restano sempre quelle: il confronto dà sempre lo stesso risultato. In realtà possiamo far dipendere una variabile da qualcosa che **succede fuori** dalla scheda, per esempio da un **pulsante** premuto oppure no.

2 Il pulsante come ingresso, il LED come uscita

Colleghiamo ad Arduino un **pulsante**. Il pulsante è un dispositivo che **manda un'informazione** ad Arduino: l'informazione va **verso la scheda**. Quindi il piedino a cui lo colleghiamo dev'essere un **ingresso** (`INPUT`). Può essere un piedino **qualunque** (il 3, il 9, il 12, quello che volete): l'importante è impostarlo come `INPUT`. Nell'esempio usiamo il **piedino 7**.

Dall'altra parte colleghiamo un **LED**, con il suo resistore, a un piedino impostato come **uscita** (`OUTPUT`): nell'esempio il **piedino 3**. Il LED poi torna al **GND**.

Nota

Il simbolo del LED. Il LED è un **diodo** che emette luce: il suo simbolo è quello del diodo (un **triangolo** con una **sbarretta**) con in più due **freccine** che escono, la luce.

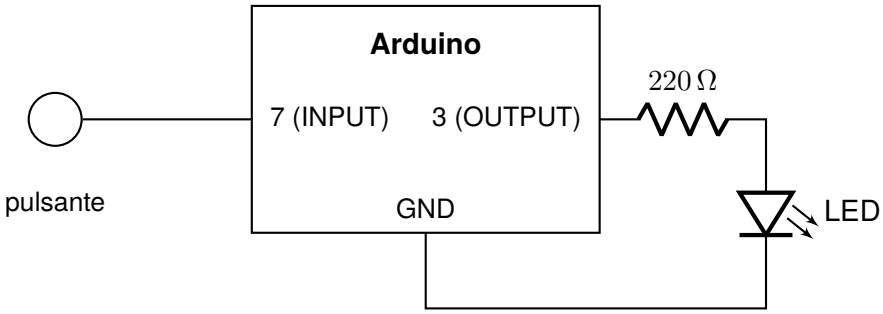


Figura 1: Lo schema disegnato alla lavagna: il pulsante manda l'informazione al piedino 7 (ingresso), il piedino 3 (uscita) accende il LED, che torna al GND. Il collegamento completo del pulsante si vede nel circuito in Tinkercad.

2.1 Il pulsante va ricontrollato a ogni giro

Arduino non "sente" il pulsante in modo continuo. Il codice dentro `loop` viene **ripetuto continuamente**: a **ogni ripetizione** Arduino deve **ricontrollare** se il pulsante è premuto oppure no.

Per leggerlo c'è il comando che abbiamo già visto: `digitalWrite` serve per **scrivere**, `digitalRead` (con la **R maiuscola**) serve per **leggere**. Tra parentesi gli passiamo il numero del piedino, qui il 7:

```
a = digitalRead(7);
```

Il valore letto (`HIGH` o `LOW`) lo salviamo in una variabile, per esempio `a`. Poi ci chiediamo: se `a` è **alto**, accendo il LED; se `a` è **basso**, lo spengo.

3 Il programma completo

Questo è il programma scritto alla lavagna.

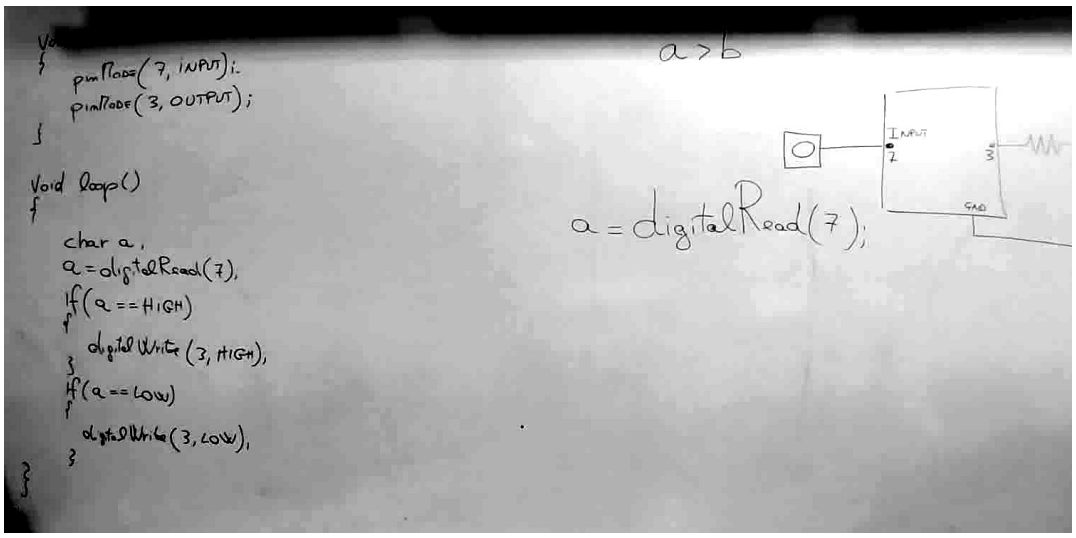


Figura 2: La lavagna: a sinistra il programma, a destra lo schema con il pulsante sul 7 e il LED sul 3.

```
void setup()
{
  pinMode(7, INPUT);
  pinMode(3, OUTPUT);
}

void loop()
{
  char a;
  a = digitalRead(7);

  if (a == HIGH)
  {
    digitalWrite(3, HIGH);
  }

  if (a == LOW)
  {
    digitalWrite(3, LOW);
  }
}
```

- In `setup`: il piedino **7** è un **ingresso** (c'è il pulsante), il piedino **3** è un'**uscita** (c'è il LED).
- In `loop`: creiamo la variabile `char a` e ci mettiamo dentro lo **stato del pulsante**, letto con `digitalRead(7)`. Con queste due righe creiamo una variabile e leggiamo il pulsante.
- Primo `if`: se `a` è **alto** (pulsante premuto), `digitalWrite(3, HIGH)` **accende** il LED.
- Secondo `if`: se `a` è **basso** (`LOW`), `digitalWrite(3, LOW)` lo **spegne**. Quando togliamo il dito dal pulsante, `digitalRead` restituisce `LOW`, quindi anche `a` diventa basso e il LED si spegne.

Dentro ciascuna coppia di graffe qui c'è **una sola istruzione**, ma potremmo metterne **tante**: tutte vengono eseguite se la condizione è vera.

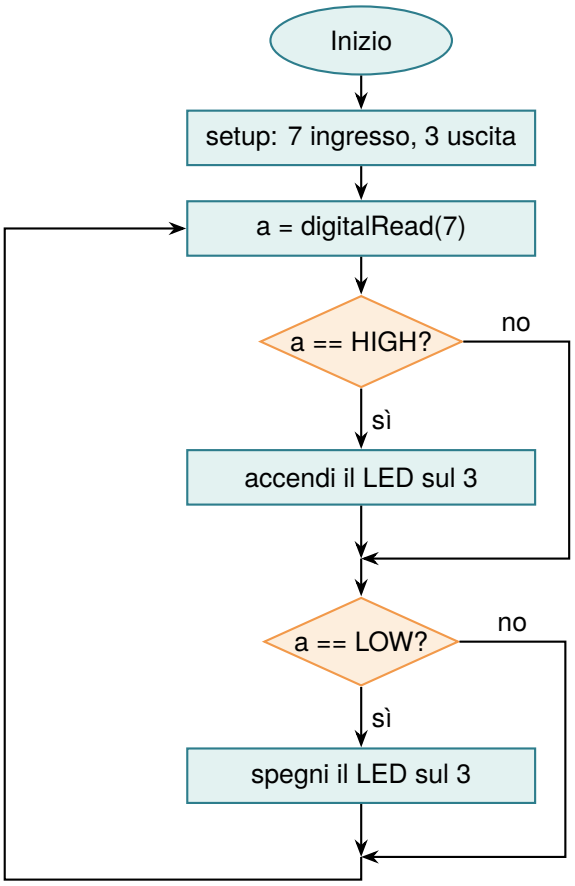


Figura 3: Il programma del pulsante: a ogni giro di `loop` Arduino legge il piedino 7 e decide se accendere o spegnere il LED. Il `loop` non finisce mai: dopo l'ultimo controllo si ricomincia dalla lettura.

4 Proviamolo in Tinkercad

4.1 Il LED

1. Ci serve un **Arduino** e una **breadboard**.
2. Colleghiamo **subito GND e 5 V** ai rail della breadboard in basso (nero al meno, rosso al più), così la piattaforma è già pronta: questi fili vanno **messi sempre**.
3. Sul **lato destro** della breadboard uniamo i rail in basso a quelli **in alto** con due fili: **nero** tra i due meno, **rosso** tra i due più. Così anche i rail in alto hanno **GND e 5 V**: il pulsante prenderà i 5 V proprio dal rail in alto.
4. Mettiamo un **LED** collegato al **piedino 3** di Arduino, come avete già fatto. Come al solito il **resistore** è da **220 Ω**, e il catodo del LED va al rail del **GND**.

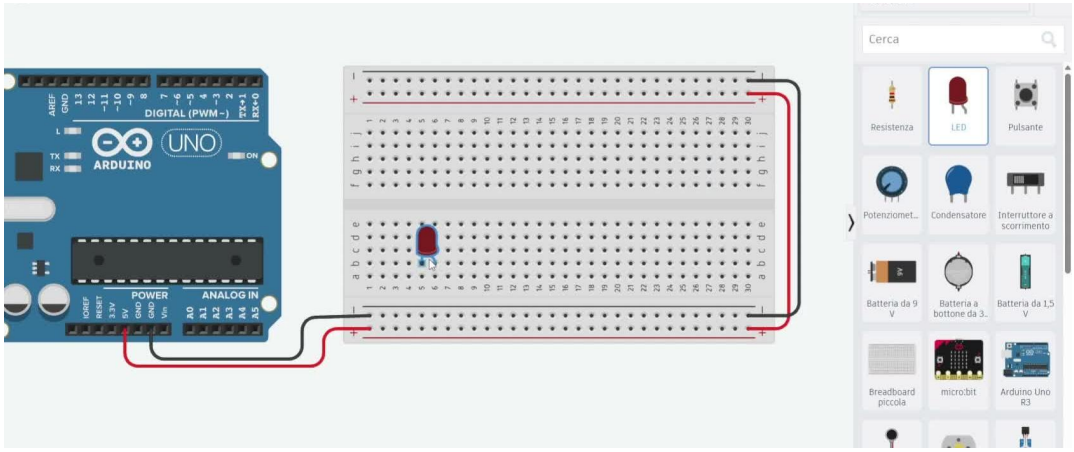


Figura 4: La piattaforma pronta: GND e 5 V di Arduino sui rail in basso, e sul lato destro due fili (nero e rosso) che portano GND e 5 V anche ai rail in alto.

4.2 Il pulsante

Il pulsante si collega così (**come** funziona questo collegamento lo vediamo in **elettronica**, lunedì):

1. si mette il **pulsante** a cavallo del **solco centrale** della breadboard (“qua in mezzo”);
2. da un piedino del pulsante si mette un **resistore** verso il rail del **GND**: lo potete lasciare da **1 kΩ**, come lo propone Tinkercad;
3. un **filo rosso** collega l’**altro lato** del pulsante (il lato destro) ai **5 V**;
4. tra il resistore e il pulsante restano **due buchi liberi**: ne scegliete **uno**, quello che volete, e da lì portate un filo al **piedino 7**. Attenzione a mettere il filo **proprio nel buco**, al centro.

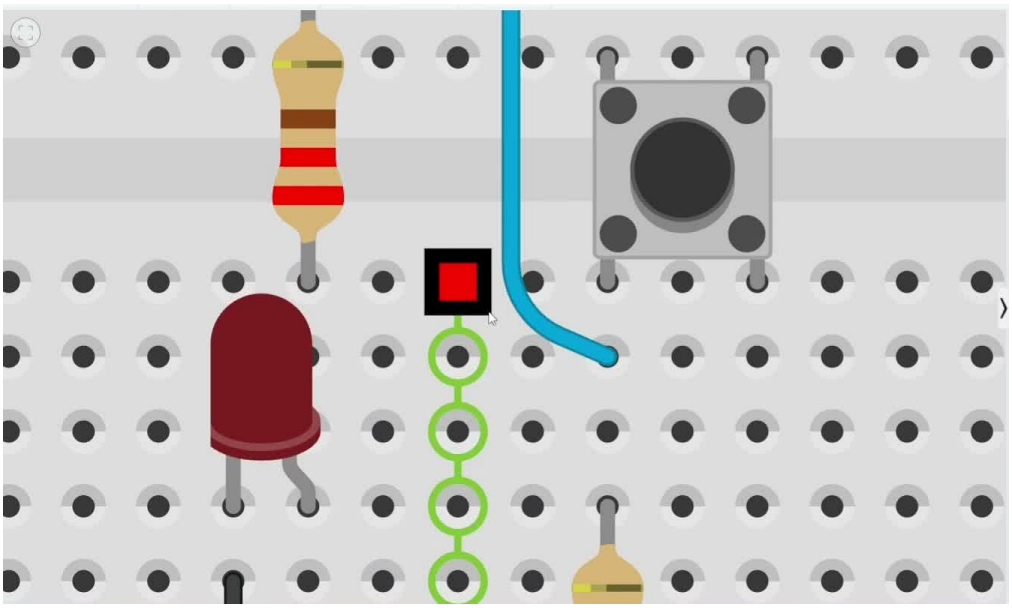


Figura 5: Il pulsante a cavallo del solco centrale; sotto, il resistore verso il GND; il filo azzurro va a uno dei buchi liberi tra pulsante e resistore.

Nota

A che cosa serve il resistore del pulsante. Quando il pulsante **non è premuto**, il piedino 7 è collegato al **GND** attraverso il resistore, quindi `digitalRead(7)` legge **LOW**. Quando lo **premete**, il pulsante collega il piedino 7 ai **5 V** e la lettura diventa **HIGH**. Senza il resistore, a pulsante rilasciato il piedino non sarebbe collegato a niente e la lettura sarebbe **casuale**. Un resistore usato così si chiama **resistore di pull-down**: lo approfondiamo in elettronica.

4.3 Il codice

Si apre il pannello **Codice**: in un progetto nuovo parte in modalità **Blocchi**, quindi si passa alla modalità **Testo**. Lì Tinkercad mette un **programma di esempio** (fa lampeggiare il LED della scheda, con `LED_BUILTIN` e `delay(1000)`): si **cancella** tutto e si **copia** il programma della lavagna. Quando avete finito, avviate la **simulazione**: premendo il pulsante il LED si deve **accendere**, togliendo il dito si deve **spegnere**.

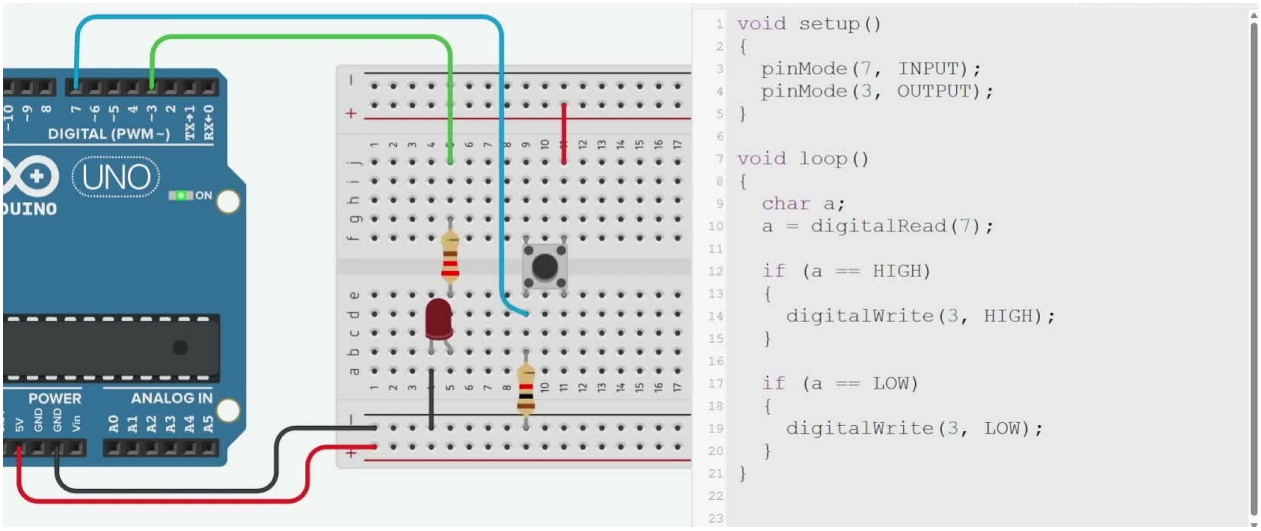


Figura 6: Il circuito finito in Tinkercad: filo verde dal piedino 3 al resistore da 220 Ω e al LED, filo azzurro dal piedino 7 al pulsante, resistore da 1 kΩ verso il GND, filo rosso dal pulsante ai 5 V. A destra il codice.

Attenzione

Le graffe sulla tastiera italiana. La graffa **aperta** { si fa con **AltGr + Maiuscolo + è**; la graffa **chiusa** } con **AltGr + Maiuscolo + +**. Se un tasto della tastiera non funziona si segnala; nel frattempo si può **copiare** una graffa già scritta (per esempio quella di riga 6) e **incollarla** dove serve.

5 Gli errori più frequenti

Quando il programma non funziona, prima di tutto si **legge il messaggio** che Tinkercad scrive **in basso**, in rosso: dice **che cosa** manca e **in quale riga**.

- **Maiuscole e minuscole**. Il C le distingue: `setup` si scrive tutto **minuscolo** (non `Setup`), `pinMode` con la **p minuscola** e la **M maiuscola** (non `PinMode`), `INPUT` e `OUTPUT` tutti **maiuscoli**, `digitalRead` con la **R maiuscola**.
- **Nome scritto male**. Se manca una lettera (per esempio `digitaRead`), il compilatore non conosce il comando: "`'digitaRead' was not declared in this scope`", cioè "non è stato dichiarato", e propone l'alternativa giusta ("`suggested alternative: 'digitalRead'`").
- **Punto e virgola mancante**. Il messaggio è "`expected ';' before '}'`": "mi aspettavo un punto e virgola prima della graffa chiusa". Il numero di riga dice dove guardare: il punto e virgola manca alla fine dell'istruzione **prima** di quella graffa.
- **Graffe**. Ogni graffa aperta deve avere la sua graffa **chiusa**; se non le chiudete tutte il programma non si può provare. Attenzione anche alla graffa "storta" (tonda o quadra al posto della graffa) e alla graffa in più che non chiude niente: il messaggio è del tipo "`expected '}' at end of input`".
- **Circuito e codice devono coincidere**. Se il LED è collegato al piedino **2** ma nel codice c'è `digitalWrite(3, ...)`, il LED non si accende: il numero nel codice deve essere **quello reale**. Lo stesso se un filo finisce nella **colonna sbagliata** della breadboard (per esempio la 6 invece della 5).
- **I fili di alimentazione** della breadboard (GND e 5 V) vanno messi **sempre**: senza, niente funziona. Se il circuito sembra giusto ma premendo il pulsante il LED non si accende, controllate il **lato destro**: senza i due fili che uniscono i rail in basso a quelli in alto, il rail da cui il pulsante prende i 5 V **non è alimentato**.
- **Niente fili volanti**: si lavora **sempre sulla breadboard**.
- Se avete spostato per sbaglio la vista o un componente, il pulsante con la **freccia verso sinistra** in alto a sinistra (**annulla**) torna indietro di un passo; poi si **riavvia** la simulazione.

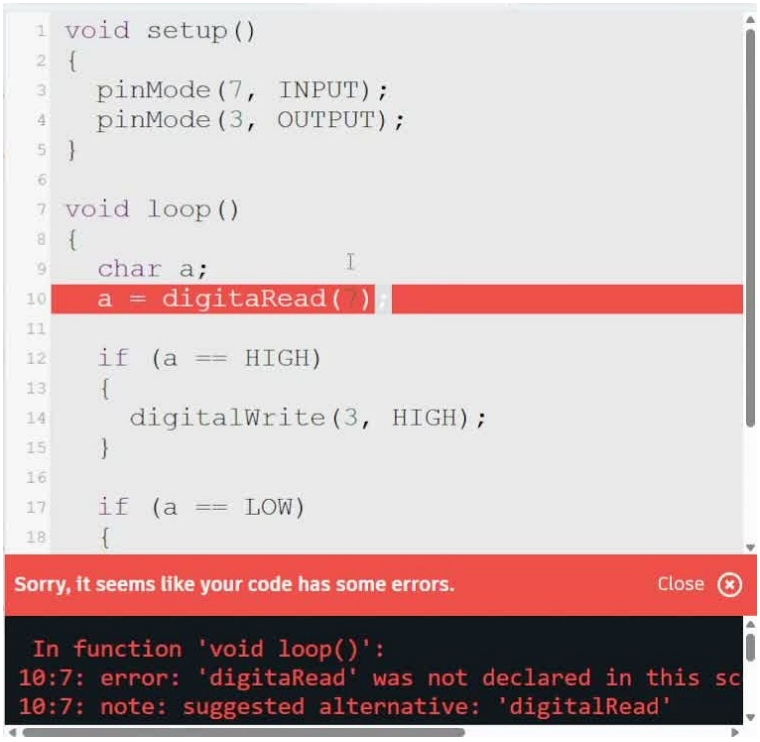


Figura 7: Un nome scritto male: la riga 10 diventa rossa e in basso compare il messaggio con il numero di riga e l'alternativa suggerita.

6 Esercizio: quattro LED e un pulsante

Attenzione

Prima un backup. Il progetto del pulsante che funziona **tenetelo**: è il vostro **riferimento** per rivedere come si scrive il codice. Per l'esercizio create un **progetto nuovo**. E prima di passare all'esercizio, il primo circuito deve **funzionare**.

Testo. Create un Arduino con collegati **quattro LED**, tutti e quattro che tornano sul **GND**, sui piedini che volete voi. Collegate poi un **pulsante** nella stessa maniera di prima.

- Se il pulsante è **premuto**, si accendono **solamente due** LED (scelti da voi).
- Se il pulsante **non è premuto**, si accendono **solamente gli altri due**.
- In entrambi i casi dovete prevedere anche lo **spegnimento** degli altri due, per essere sicuri che si spengano.

Bastano **due if**, non di più.

6.1 Una soluzione

Nella soluzione fatta in aula i LED sono sui piedini **2** (rosso), **3** (verde), **4** (rosso) e **5** (verde); il pulsante è sul piedino **8**. Pulsante premuto: si accendono i **due rossi** e si spengono i verdi. Pulsante rilasciato: il contrario.

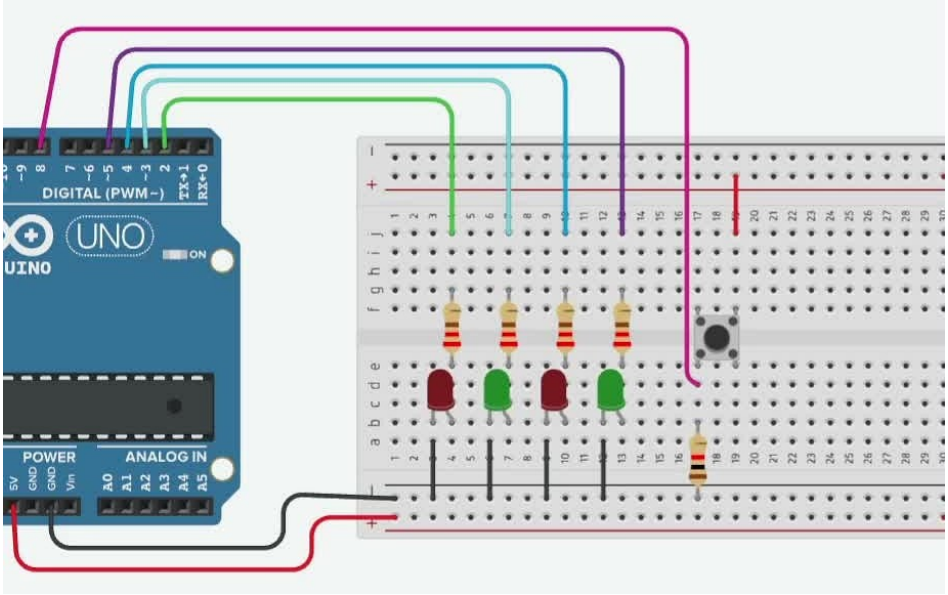


Figura 8: Quattro LED sui piedini 2, 3, 4, 5 (rosso, verde, rosso, verde), ciascuno con il suo resistore da 220 Ω; il pulsante sul piedino 8 (filo rosa), collegato come prima.

```
void setup()
{
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(8, INPUT);
}

void loop()
{
  char a;
  a = digitalRead(8);

  if (a == HIGH)
  {
    digitalWrite(2, HIGH);
    digitalWrite(3, LOW);
    digitalWrite(4, HIGH);
    digitalWrite(5, LOW);
  }

  if (a == LOW)
  {
    digitalWrite(2, LOW);
    digitalWrite(3, HIGH);
    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
  }
}
```

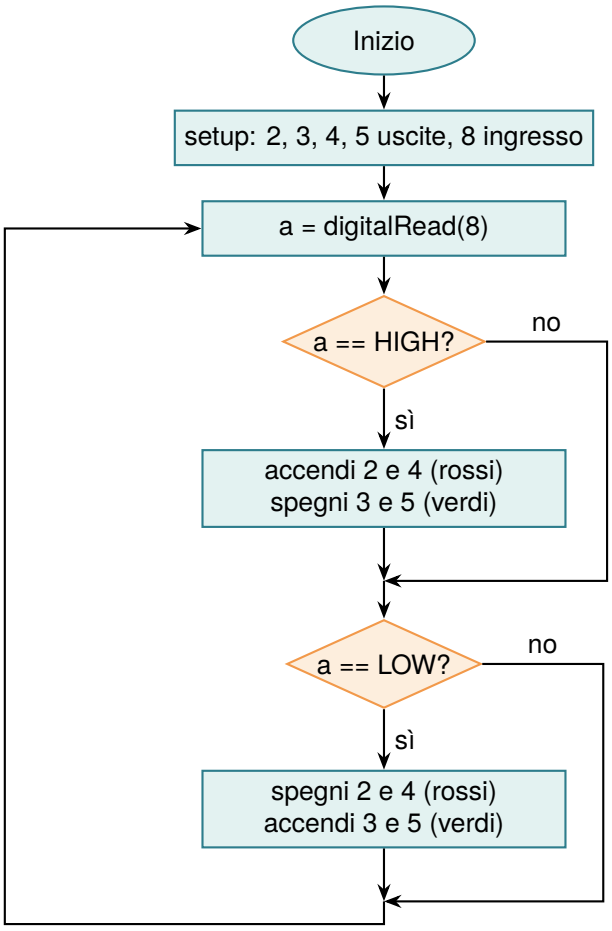


Figura 9: L'esercizio dei quattro LED: due if, uno per il pulsante premuto e uno per il pulsante rilasciato; in ognuno si accendono due LED e si spengono gli altri due.

Nota

Perché spegnere sempre gli altri due. Un `digitalWrite(..., HIGH)` lascia il LED acceso finché qualcuno non scrive `LOW`. Se nel primo if accendiamo i rossi ma non spegniamo i verdi, i verdi rimasti accesi dal giro precedente **restano accesi**: per questo in ogni if si scrivono **tutti e quattro** i LED.

7 Cosa ricordare di questa lezione

- `if (condizione) { ... }`: se la condizione è **vera** esegue il blocco tra le graffe, se è **falsa** lo **salta**.
- Una variabile può dipendere dall'**esterno**: `a = digitalRead(7);` legge un **pulsante**.
- Il piedino del pulsante è un **INPUT**, quello del LED un **OUTPUT**.
- Il `loop` si ripete: a **ogni giro** il pulsante viene **riletto**.
- Il pulsante in Tinkercad: a cavallo del solco, **resistore da 1 kΩ** verso il GND, **5 V** dall'altro lato, **filo al piedino** tra pulsante e resistore.

- Quando qualcosa non va, si **legge il messaggio** in basso: dice cosa manca e in quale riga. Attenzione a **maiuscole**, **punti e virgola**, **graffe** e ai **numeri dei piedini**.
- Graffe sulla tastiera italiana: **AltGr + Maiuscolo + è** e **AltGr + Maiuscolo + +**.
- Con quattro LED: **due if**, e in ognuno si decide lo stato di **tutti** i LED.